



Teste Prático - Desenvolvedor Backend | Nexus

Você vai construir uma API REST de uma carteira cripto simplificada.

O objetivo é avaliar como você estrutura código, modela dados e resolve problemas reais do nosso dia a dia.

Prazo: 7 dias

Entrega: repositório no GitHub (público ou privado com acesso)

STACK OBRIGATÓRIA:

- Node.js + TypeScript
- PostgreSQL (com ORM de preferência — sugerimos Prisma)
- Git

STACK OPCIONAL:

- NestJS ou Fastify (frameworks de preferência da equipe)
- Zod (validação de dados)
- Redis

REQUISITOS OBRIGATÓRIOS

1. AUTENTICAÇÃO

- Cadastro de usuário (email + senha)
- Login retornando JWT (access token + refresh token)
- Rotas protegidas por middleware de autenticação

2. CARTEIRA E SALDOS

- Ao se cadastrar, o usuário recebe uma carteira com saldo zero
- A carteira suporta pelo menos 3 tokens: BRL, BTC e ETH
- Saldos armazenados no banco (modelo de ledger virtual)
- Endpoint para consultar saldos da carteira

3. DEPÓSITO VIA WEBHOOK

- Endpoint POST /webhooks/deposit que simula um serviço externo notificando um depósito
- Payload: { userId, token, amount, idempotencyKey }



- Validar idempotencyKey para evitar depósitos duplicados
- Creditar o saldo do usuário no token correto
- Retornar erro se usuário ou token não existir

4. SWAP (CONVERSÃO ENTRE TOKENS)

- Endpoint para cotar um swap (ex: quanto custa 0,5 BTC em BRL?)
 - Usar a API pública da CoinGecko ou similar para obter cotação real
 - Aplicar uma taxa fixa de 1,5% sobre o valor
 - Retornar: quantidade de destino, taxa cobrada, cotação usada
- Endpoint para executar o swap
 - Validar se o usuário tem saldo suficiente (incluindo taxa)
 - Debitar token de origem + taxa
 - Creditar token de destino
 - Registrar a transação

5. SAQUE

- Endpoint para solicitar saque de um token
- Validar se o usuário tem saldo suficiente
- Debitar o saldo (a transferência em si é mock — não precisa integrar com blockchain ou banco real)
- Registrar a transação de saque

6. LEDGER (MOVIMENTAÇÕES)

- Toda alteração de saldo deve gerar um registro de movimentação
- Cada movimentação registra: tipo (DEPOSIT, SWAP_IN, SWAP_OUT, SWAP_FEE, WITHDRAWAL), token, valor, saldo anterior, saldo novo, data/hora
- O saldo atual do usuário deve poder ser reconstruído a partir das movimentações (auditável)
- Endpoint para consultar extrato de movimentações com paginação

Exemplo de fluxo completo:

1. Depósito de 10 USDT → mov: +10 USDT (saldo: 0 → 10)
2. Swap 10 USDT → BRL (55 BRL) →
 - mov: -10 USDT (10 → 0)
 - mov: -0,825 BRL taxa (0 → -0,825)
 - mov: +55 BRL (-0,825 → 54,175)



3. Saque de 50 BRL → mov: -50 BRL (54,175 → 4,175)

Extrato mostra todas as 5 movimentações em ordem.

7. HISTÓRICO DE TRANSAÇÕES

- Endpoint para listar transações do usuário
- Cada transação registra: tipo (DEPOSIT, SWAP ou WITHDRAWAL), tokens envolvidos, valores, taxa (se houver), data/hora
- Suportar paginação

8. README

- Como rodar o projeto localmente
- Decisões técnicas relevantes (por que escolheu X ao invés de Y)
- Estrutura do banco de dados (pode ser diagrama ou descrição)

DIFERENCIAIS (NÃO OBRIGATÓRIOS)

Estes itens não são exigidos, mas contam pontos extras na avaliação:

- Redis para cache de cotações (evitar chamadas repetidas à API externa)
- Front-end integrado em React consumindo a API
- Deploy funcional (qualquer plataforma: Railway, Render, VPS, etc.)

O QUE VAMOS AVALIAR

- Modelagem do banco de dados e relacionamentos
- Organização e clareza do código (módulos, separação de responsabilidades)
- Ledger de movimentações (rastreabilidade, consistência dos saldos)
- Tratamento de erros e casos de borda (ex: saldo insuficiente, depósito duplicado)
- Uso correto de TypeScript (tipagem, interfaces, DTOs)
- Qualidade do README e documentação de decisões
- Bônus pelos diferenciais entregues

DICA: Prefira entregar o core bem feito do que tentar fazer tudo de forma superficial.
Qualidade > Quantidade.